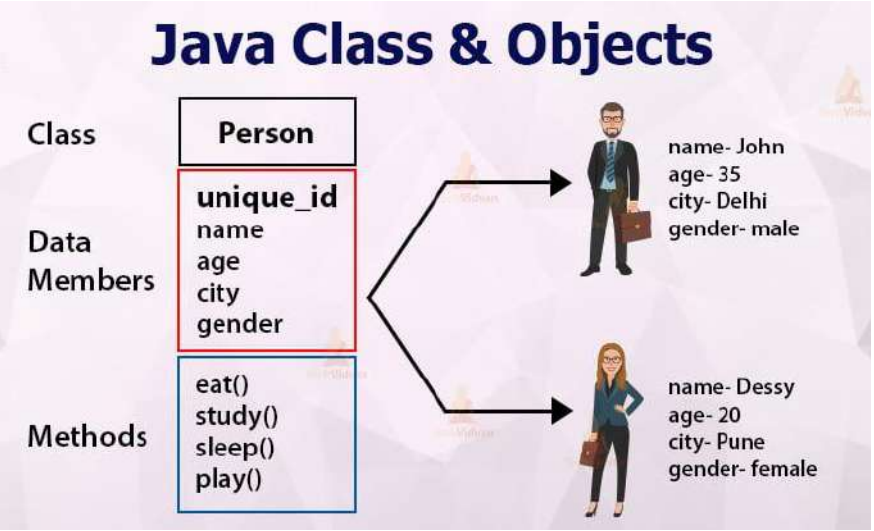
Theory /Explaination/content/**Revision**

Content (**OOPS Implementation**)

1. Classes,
2. Objects,
3. attributes,
4. methods,
5. data encapsulation,
6. reference variables,
7. Constructors,
8. Anonymous block,
9. Method Overloading,
10. Static Data members, Block & methods;
11. Memory Structure: Stack, Heap, Class & Method area
12. Class loading & Execution flow: Static vs Dynamic Class loading, implicit vs explicit class loading, class loading operations;
13. Argument Passing Mechanism: Passing primitive arguments, passing objects,
14. Wrapper Classes;
15. Packages and Access Modifiers

1. Classes (5 Marks)

A **class** is a fundamental concept in object-oriented programming (OOP) that serves as a blueprint for creating objects.
It defines the attributes (data members) and behaviors (methods) that the objects will have.
A class does not occupy memory until an object is instantiated from it.



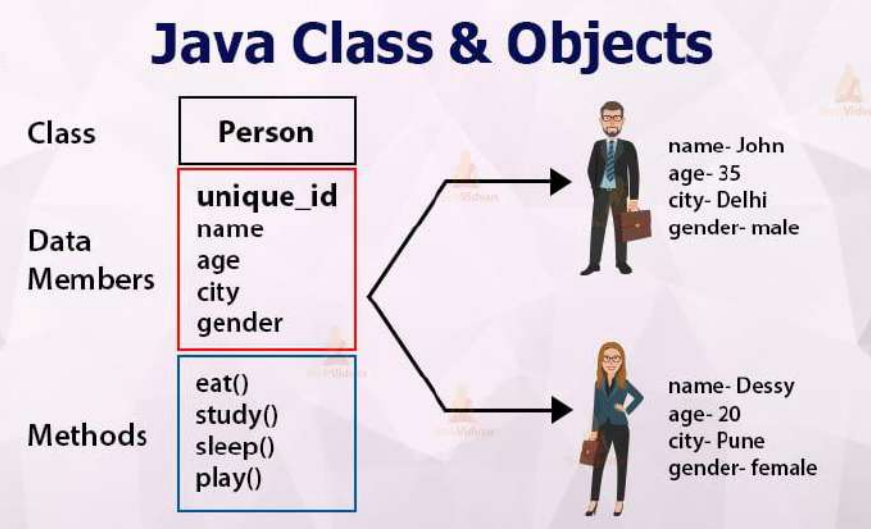
Key Points:

- A class represents a **logical entity** rather than a physical one.
- It allows for **data abstraction and organization** of code.
- It consists of **attributes (variables)** and **methods (functions)** that define the behavior of objects.
- A class promotes **code reusability** as multiple objects can be created from the same class.

For example, in a real-world scenario, a **"Car"** can be considered a class, with attributes like **color, model, and speed**, and methods like **start(), stop(), and accelerate()**.

2. Objects (5 Marks)

An **object** is an instance of a class. It is a real-world entity that has a **state (attributes)** and **behavior (methods)**. Objects are created from a class, and each object has its own distinct identity in memory.



Key Points:

- Objects are **runtime entities** that occupy memory.
- They **store data** in attributes and **use methods** to perform actions.
- Each object can have **different values** for the same attributes defined in the class.
- Objects are used to **interact with and manipulate data** in OOP.

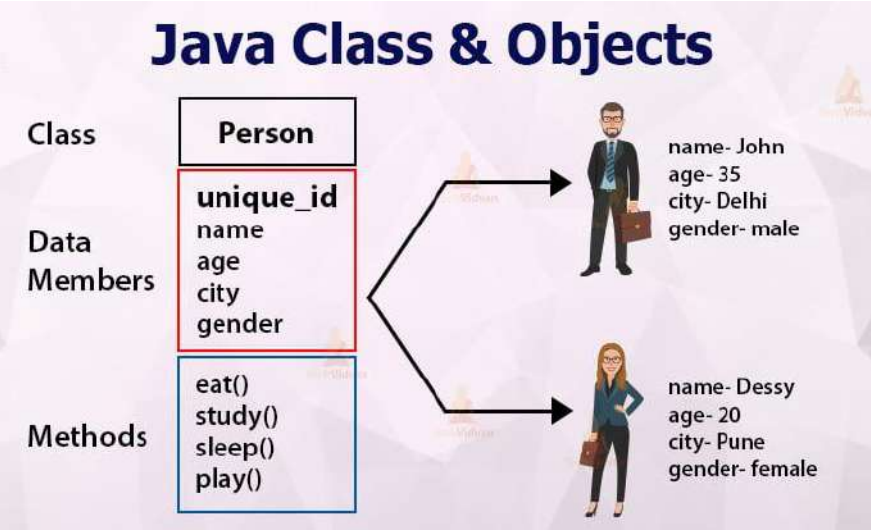
For example, if **"Car"** is a class, then:

- **Car1** (Red, Toyota, 120 km/h)
- **Car2** (Blue, Honda, 140 km/h)

Both are objects with different attribute values but the same class structure.

3. Attributes (5 Marks)

Attributes are **variables declared inside a class** that define the properties or characteristics of an object. They are also known as **fields, instance variables, or data members**.



Key Points:

- Each object has its own **copy** of the attributes.
- Attributes store **object-specific data**, defining its **state**.
- They can have **different access modifiers** (public, private, protected) to control th visibility.
- Attributes can be initialized using **constructors or methods**.

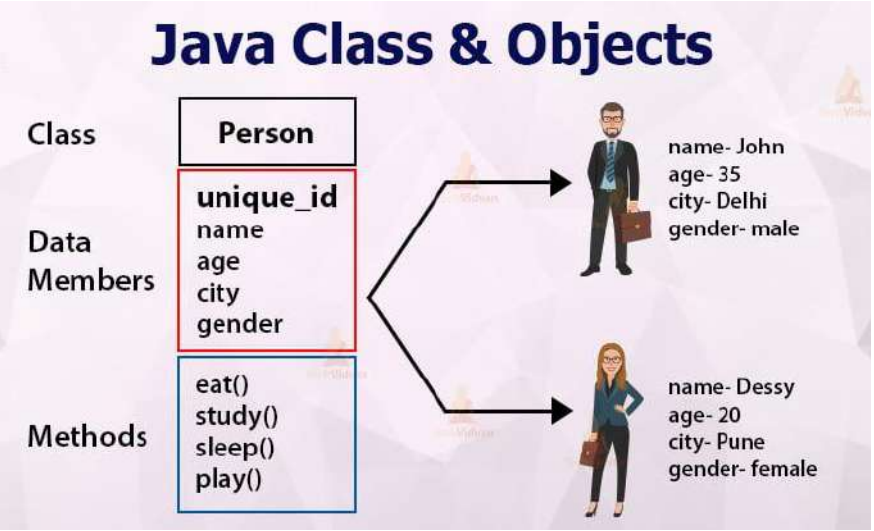
For example, in a **Student** class, attributes can be:

- **name** (String)
- **age** (Integer)
- **grade** (Character)

Each student object will have different values for these attributes.

4. Methods (5 Marks)

A **method** is a function defined inside a class that operates on an object's attributes and provides behavior to the class. Methods are used to perform actions on object data.



Key Points:

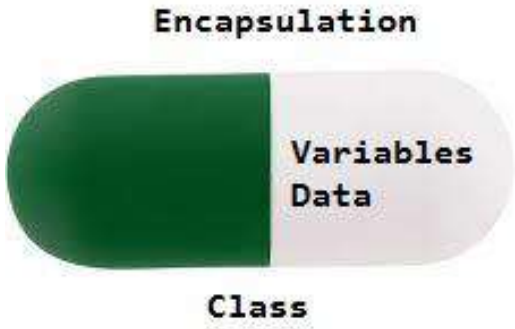
- Methods allow **code reusability** by defining common operations.
- They can be of different types: **instance methods, static methods, constructors, and accessors**.
- Methods are called using an object to execute a task.
- They can take **parameters** and return **values**.

For example, in a **BankAccount** class, methods can be:

- **deposit(amount)** – Adds money to the account.
- **withdraw(amount)** – Deducts money from the account.
- **checkBalance()** – Returns the account balance.

5. Data Encapsulation (5 Marks)

Encapsulation is one of the fundamental principles of OOP that restricts direct access to certain details of an object and only allows controlled access through methods. It is achieved using **private attributes and public getter/setter methods**.



Key Points:

- It **hides the implementation details** from outside users.
- Prevents **unauthorized access** to sensitive data.
- Ensures **data security** and **modular code** structure.
- Uses **getters and setters** to access private data.

For example, in a **BankAccount** class:

- The **balance** attribute is private.
- Methods like **getBalance()** and **setBalance()** allow controlled access.

6. Reference Variables (5 Marks)

A **reference variable** is a variable that holds the memory address of an object rather than the actual object itself.

<u>It acts as a pointer to an object in memory.</u>

Key Points:

- It is used to **access and manipulate objects** in memory.
- Multiple reference variables can **point to the same object**.
- Assigning a reference variable to another does not create a new object but shares the reference.
- Helps in **efficient memory management** in OOP.

Car c1 = new Car(); // **c1 is a reference variable pointing to a Car object**

Car c2 = c1; // **c2 now also refers to the same Car object**

7. Constructors (5 Marks)

A constructor is a special method in a class that is automatically called when an object is created.
It is used to initialize objects with default or user-defined values.

Key Points:

- A constructor has the same name as the class.
- It does not have a return type (not even void).
- It can be parameterized or non-parameterized.
- If no constructor is defined, a default constructor is provided by the compiler.

For example, in a Person class:

```
Person(String name, int age)
{
  this.name = name;
  this.age = age;
}
```

This constructor initializes **name** and **age** when an object is created.

8. Anonymous Block (5 Marks)

An **anonymous block** (also known as an instance initializer block) is a block of code **inside a class but outside any method or constructor**.
It is automatically executed when an object is created, before the constructor is called.

Key Points:

- Used for **initialization of instance variables**.
- Runs **before the constructor** when an object is created.
- Can be used to execute **common code for all constructors**.

For example:

```
class Test {
  {
    System.out.println("Anonymous Block Executed");
  }
}
```

Every time an object of Test is created, the anonymous block executes before the constructor.

Anonymous Block

```
class Demo {
    // Anonymous block
    {
        System.out.println("Anonymous block executed");
    }

    // Constructor
    Demo() {
        System.out.println("Constructor executed");
    }
}

public class Program{
    public static void main(String[] args) {
        Demo obj = new Demo();
    }
}
```

```
Anonymous block executed
Constructor executed
```

Explanation: The anonymous block runs **before** the constructor.

9. Method Overloading (5 Marks)

Method overloading allows multiple methods in a class to have the **same name** but with **different parameters** (different number, type, or order of arguments).

Key Points:

- Increases **code readability and reusability**.
- Helps in **polymorphism** by allowing different ways to perform a task.
- The return type **can be different** but is not considered for overloading.

For example, in a **MathOperations** class:

```
int add(int a, int b)
double add(double a, double b)
int add(int a, int b, int c)
```

All these methods are overloaded versions of add().

Method Overloading

```
class MathOperations {
   int add(int a, int b) {
      return a + b;
   }

   double add(double a, double b) {
      return a + b;
   }
}

public class Program{
   public static void main(String[] args) {
      MathOperations obj = new MathOperations();
      System.out.println("Sum (int): " + obj.add(5, 10));
      System.out.println("Sum (double): " + obj.add(5.5, 2.5));
   }
}
```

```
Sum (int): 15
Sum (double): 8.0
```

10. Static Data Members (5 Marks)

A static data member is a class-level variable that is shared among all instances of the class.
It belongs to the class rather than individual objects.

Key Points:

- Declared using the static keyword.
- Memory is allocated only once, reducing memory usage.
- Can be accessed using the class name without creating an object.

For example:

```
class Employee {
   static int count = 0; // Static variable
}
```

count is shared among all Employee objects.

11. Static Blocks & Methods (5 Marks)

Blocks are sections of code enclosed within {}.

They include:

- Instance initializer blocks (executed before constructors).
- Static blocks (executed once when the class is loaded).

Methods define behaviors and include:

- Instance methods (called on objects).
- Static methods (called on the class).

Example:

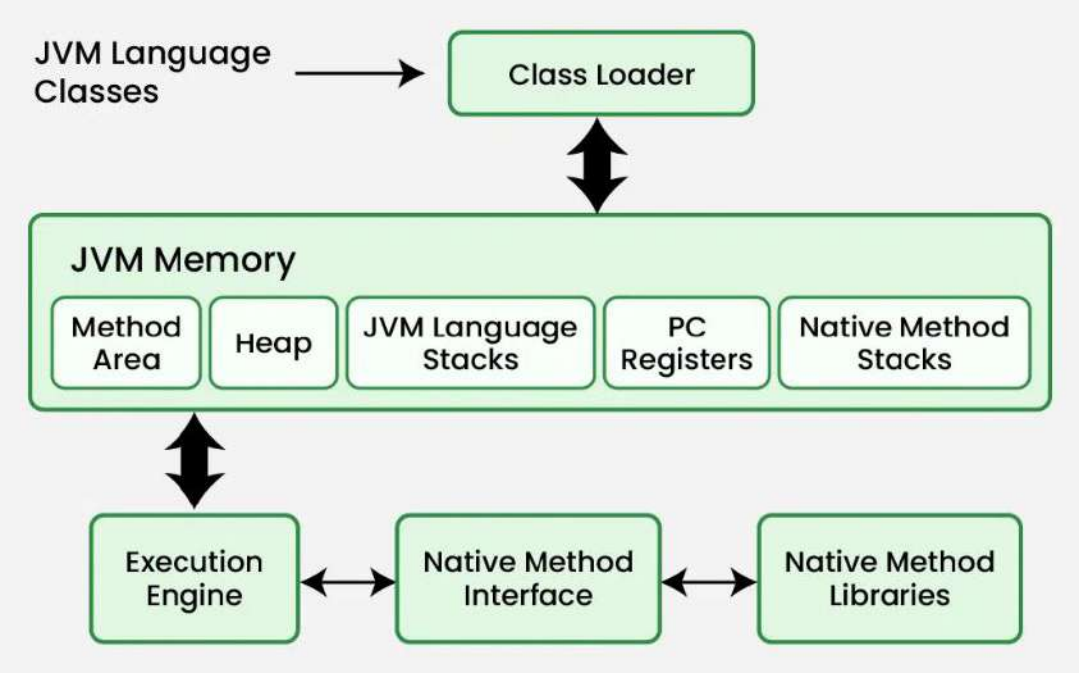
```
static { System.out.println("Static Block Executed"); }
```

```java
class Example {
    static int count = 0;  // Static variable

    // Static block
    static {
        System.out.println("Static block executed");
    }

    Example() {
        count++;
    }
// Static method
    static void showCount() {
        System.out.println("Number of objects created: " + count);
    }
}
public class Program {
    public static void main(String[] args) {
        Example obj1 = new Example();
        Example obj2 = new Example();
        Example.showCount();  // Calling static method
    }
}
```

Explanation:
Static block runs once when the class is loaded.
Static method showCount() is called using the class name.

```
Static block executed
Number of objects created: 2
```

Memory Structure in Java (7or 10 Marks Answer)



Memory Structure in Java (7or 10 Marks Answer)

Java's memory structure is divided into different areas to manage the execution of programs efficiently.
The key memory areas in Java are **Stack, Heap, Class Area, and Method Area**.

Stack Memory

The **Stack Memory** is used for storing method-specific data, including:

- **Method calls (function calls)**
- **Local variables** of methods
- **Reference variables** (pointing to objects in the heap)

Characteristics of Stack Memory:

✔ Organized in **Last-In-First-Out (LIFO)** order
✔ Each thread has its own **separate stack**
✔ Automatically allocated and deallocated when a method is called and completed
✔ Fast access but **limited in size**

```
void example() {
   int x = 10; // Stored in stack
   String name = "John"; // Reference stored in stack, object in
heap
}
```

Here, x and name are stored in the stack, while the actual "John" object is in the heap.

Memory Structure in Java (10 Marks Answer)

Java's memory structure is divided into different areas to manage the execution of programs efficiently.
The key memory areas in Java are **Stack, Heap, Class Area, and Method Area**.

Heap Memory

The **Heap Memory** is used for storing objects and class instances. It is the largest memory area and supports **dynamic memory allocation**.

Characteristics of Heap Memory:

✔️ Stores **objects and instance variables**
✔️ **Shared among all threads**
✔️ Objects remain in the heap **until garbage collected**
✔️ Slower than stack memory but **flexible in size**

Example:

```
class Car {
    String brand;
}

public class Main {
    public static void main(String[] args) {
        Car c1 = new Car(); // c1 is in stack, Car object is in heap
    }
}
```

Here, c1 is a reference variable in the stack, while the Car object is created in the heap.

Memory Structure in Java (10 Marks Answer)

Java's memory structure is divided into different areas to manage the execution of programs efficiently.
The key memory areas in Java are **Stack, Heap, Class Area, and Method Area**.

Class Area (Method Area)

The **Class Area** (part of the Method Area) stores metadata related to classes and static members.

Characteristics of Class Area:

✔ Stores **class-level data**, including:

- **Class metadata** (names, structures)
- **Static variables**
- **Static methods**

✔ **Shared across all threads**

✔ Created when the class is loaded into memory

```
class Example {
    static int count = 0; // Stored in Method Area
}
```

Here, count is a static variable and is stored in the Method Area.

Memory Structure in Java (10 Marks Answer)

Java's memory structure is divided into different areas to manage the execution of programs efficiently.
The key memory areas in Java are **Stack, Heap, Class Area, and Method Area**.

Method Area

The **Method Area** stores compiled code and class-level information required for execution.
It is part of the **JVM's runtime memory**.

Characteristics of Method Area:

✔ Stores:

- **Compiled bytecode of methods**
- **Method information (name, return type, parameters)**
- **Static methods**

✔ **Shared among all threads**

✔ Helps in **method execution**

Example:

When a method is executed, its bytecode is **fetched from the method area** and **processed in the stack**.

This memory structure ensures **efficient program execution** and **garbage collection** for automatic memory management.

Memory Area	Stores	Access Type	Scope
Stack	Local variables, method calls	Fast	Method-level
Heap	Objects, instance variables	Slower	Application-wide
Class Area	Class metadata, static variables	Shared	Class-wide
Method Area	Compiled method code, static methods	Shared	JVM-wide

Class Loading & Execution Flow in Java

Class loading in Java is the process of bringing a class file into memory when required.

The **JVM (Java Virtual Machine)** is responsible for loading classes dynamically during runtime.

The execution flow of a Java program depends on how and when classes are loaded.

Class Loading in Java

A Java class is loaded when:

- An object of the class is created.
- A static method or static variable is accessed.
- A subclass of the class is loaded.

Steps in Class Loading:

2. **Loading:** The bytecode of a class is read from .class files and stored in the method area.
3. **Linking:** The loaded class is verified and prepared for execution. This step consists of:
   - **Verification:** Ensures the class file follows JVM standards.
   - **Preparation:** Allocates memory for static variables.
   - **Resolution:** Replaces symbolic references with actual memory addresses.
4. **Initialization:** Executes static initializers and assigns values to static variables.

2. Static vs. Dynamic Class Loading

Static Class Loading

- Happens at compile-time.
- The class is loaded automatically by the JVM.
- Uses the new keyword to create an instance.

```
class Example {
   void display() {
      System.out.println("Static Loading Example");
   }

   public static void main(String[] args)
{
   Example obj = new Example(); // Class is loaded at compile-time
    obj.display();
   }
}
```

Here, the class Example is loaded statically when the program starts.

Dynamic Class Loading

- Happens at runtime.
- Used when the class name is unknown at compile-time.
- Uses Reflection API (Class.forName() method).
- Helpful in plugins, JDBC drivers, and frameworks.

```
class Example {
   void display() {
      System.out.println("Dynamic Loading Example");
   }

   public static void main(String[] args) throws Exception {
      Class<?> cls = Class.forName("Example"); // Class is loaded dynamically
      Object obj = cls.newInstance();
      Example ex = (Example) obj;
      ex.display();
   }
}
```

Here, the Example class is loaded dynamically at runtime using Class.forName().

Implicit vs. Explicit Class Loading

Implicit Class Loading

- Automatically handled by the **JVM**.
- Happens when the class is **referenced directly** in the program.
- Uses **static loading**.

Explicit Class Loading

- Manually controlled by the developer.
- Uses dynamic class loading (Class.forName()).
- Useful for loading external libraries or plugins dynamically.

```
class Car {
    void start() {
        System.out.println("Car is starting...");
    }
}

public class Main {
    public static void main(String[] args) {
        Car c = new Car(); // Implicit class loading
        c.start();
    }
}
```

```
Class<?> c = Class.forName("Car"); // Explicitly loading the Car
class
Object obj = c.newInstance();
```

Class Loading Operations

JVM Class Loaders:

The JVM has different class loaders to load classes:

1. **Bootstrap ClassLoader:** Loads core Java classes (java.lang.*, java.util.*).
2. **Extension ClassLoader:** Loads classes from the ext directory (javax.*).
3. **Application ClassLoader:** Loads user-defined classes from the classpath.

Class Loading Execution Steps:

1. Check if the class is already loaded.
2. Load the class if not already loaded.
3. Verify and prepare the class (linking process).
4. Initialize the class (assign values to static variables and execute static blocks).

Static vs. Dynamic Class Loading

```
class Static Loading {
   static {
      System.out.println("Static Block: Static Loading class is loaded.");
   }

   public Static Loading() {
      System.out.println("Constructor: Static Loading object created.");
   }
}

class Dynamic Loading {
   static {
      System.out.println("Static Block: Dynamic Loading class is loaded.");
   }

   public Dynamic Loading() {
      System.out.println("Constructor: Dynamic Loading object created.");
   }
}
```

```
Main Method Started
Static Block: StaticLoading class is loaded.
Constructor: StaticLoading object created.
Static Block: DynamicLoading class is loaded.
Constructor: DynamicLoading object created.
Main Method Ended
```

```java
public class ClassLoadingExample
{
    public static void main(String[] args)
    {
        System.out.println("Main Method Started");

        // Static Class Loading
        StaticLoading staticObj = new StaticLoading();

        try {
            // Dynamic Class Loading
            Class<?> cls = Class.forName("DynamicLoading"); // Loads the class at runtime
            Object obj = cls.getDeclaredConstructor().newInstance();
        }
        catch (Exception e) {
            e.printStackTrace();
        }

        System.out.println("Main Method Ended");
    }
}
```

This demonstrates class loading stages:

- ✅ Static loading via new keyword
- ✅ Dynamic loading via Class.forName()
- ✅ Execution of static blocks first, then constructors

Argument Passing Mechanism in Java (7 or 10 Marks Answer)

In Java, arguments (method parameters) are passed in two ways:

1. Passing Primitive Arguments (Call by Value)
2. Passing Object Arguments (Reference Handling in Methods)

1. Passing Primitive Arguments (Call by Value)

When primitive data types (e.g., int, double, char, boolean) are passed as arguments to a method, only a copy of the value is passed. This means:

✔ The original variable remains unchanged.

✔ Any modifications inside the method affect only the copied value.

Example:

```
class Test {
   void modify(int num) {
      num = num + 10; // Change is made to local copy
      System.out.println("Inside method: " + num);
   }

   public static void main(String[] args)
 {
      int value = 50;
      Test obj = new Test();
      obj.modify(value);
      System.out.println("After method call: " + value);
// Original remains unchanged
   }
}
```

Inside method: 60
After method call: 50

Argument Passing Mechanism in Java (7 or 10 Marks Answer)

In Java, arguments (method parameters) are passed in two ways:

1. Passing Primitive Arguments (Call by Value)
2. Passing Object Arguments (Reference Handling in Methods)

Passing Object Arguments (Reference Handling in Methods)

When an object is passed as an argument, the **reference (memory address) is passed by value**, not the actual object.

This means:

✔ **The reference variable is copied**, but both the original and copied reference point to the **same object in heap memory**.

✔ Changes made inside the method **affect the original object**.

```
class Person {
    String name;

    void changeName(Person p) {
        p.name = "Alice"; // Modifying the object, not the reference
    }

    public static void main(String[] args) {
        Person person1 = new Person();
        person1.name = "John";

        System.out.println("Before: " + person1.name);
        person1.changeName(person1);
        System.out.println("After: " + person1.name); // Original
object is modified
    }
}
```

```
Before: John
After: Alice
```

Aspect	**Passing Primitive**	**Passing Object**
What is Passed?	A **copy** of the value	A **copy of the reference**
Effect on Original?	No change	Changes affect the original object
Stored in?	**Stack Memory**	Reference in **Stack**, Object in **Heap**
Used For?	int, double, char, etc.	String, Array, Custom Objects

Java always follows **call-by-value**:

- For **primitives**, only a copy is passed (original remains unchanged).
- For **objects**, a reference is passed (changes affect the original object).

This mechanism ensures **memory safety** while allowing objects to be modified within methods.

Wrapper Classes in Java (7 Marks)

1. Introduction
2. Why to use wrapper class
3. Wrapper Classes for Primitive Types
4. Autoboxing & Unboxing
   - Autoboxing (Primitive → Object)
   - Unboxing (Object → Primitive)
5. Example: Wrapper Class Usage
6. Utility Methods in Wrapper Classes
   - Parsing Strings to Primitives
   - Converting Primitives to Strings
   - Finding Min/Max Values
7. Advantages of Wrapper Classes

Wrapper Classes in Java (7 Marks)

1.Introduction

Wrapper classes in Java are used to convert **primitive** data types **into objects**.
Each primitive type has a corresponding wrapper class in the java.lang package.

2. Why Use Wrapper Classes?

✔ Enable use of primitives in collections (like ArrayList<Integer>, HashMap<Double, String>, etc.)
✔ Support autoboxing and unboxing (automatic conversion between primitives and objects)
✔ Provide utility methods for parsing, comparison, and conversions

Wrapper Classes in Java (7 Marks)

3. Wrapper Classes for Primitive Types

Primitive Type	**Wrapper Class**
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Wrapper Classes in Java (7 Marks)

4. Autoboxing & Unboxing

Autoboxing (Primitive → Object)
Java automatically converts a primitive value to its corresponding wrapper class object.

```
int num = 10;
Integer obj = num; // Autoboxing
```

Unboxing (Object → Primitive)
Java automatically converts a wrapper object back to its primitive type.

```
Integer obj = 20;
int num = obj; // Unboxing
```

Wrapper Classes in Java (7 Marks)

5. Example: Wrapper Class Usage

```
public class WrapperExample {
    public static void main(String[] args) {
        Integer intObj = Integer.valueOf(100); // Explicit conversion
        int num = intObj.intValue(); // Unboxing

        Double doubleObj = 25.5; // Autoboxing
        double value = doubleObj; // Unboxing

        System.out.println("Integer Object: " + intObj);
        System.out.println("Primitive int: " + num);
        System.out.println("Double Object: " + doubleObj);
        System.out.println("Primitive double: " + value);
    }
}
```

```
Integer Object: 100
Primitive int: 100
Double Object: 25.5
Primitive double: 25.5
```

Wrapper Classes in Java (7 Marks)

6 Utility Methods in Wrapper Classes

Wrapper classes provide various utility methods, such as:

Parsing Strings to Primitives

```
int num = Integer.parseInt("123"); // Converts string "123" to int
double d = Double.parseDouble("45.67"); // Converts string "45.67" to double
```

Converting Primitives to Strings

```
String str = Integer.toString(500); // Converts int to string "500"
```

Finding Min/Max Values

```
System.out.println(Integer.MAX_VALUE); // 2147483647
System.out.println(Integer.MIN_VALUE); // -2147483648
```

Wrapper Classes in Java (7 Marks)

7. Advantages of Wrapper Classes

✔ Allow primitives to be used in collections (like ArrayList, HashMap, etc.)
✔ Provide utility methods for conversion, parsing, and comparison
✔ Support autoboxing/unboxing for easy conversion between primitive and object types

Access Modifiers

Modifier	Same Class	Same Package	Different Package (Non-Subclass)	Different Package (Subclass)
public	✅ Yes	✅ Yes	✅ Yes	✅ Yes
private	✅ Yes	❌ No	❌ No	❌ No
protected	✅ Yes	✅ Yes	❌ No	✅ Yes (only via subclass)
default (no modifier)	✅ Yes	✅ Yes	❌ No	❌ No

How public, private, protected, and default (no modifier) affect package visibility in Java.

Public (public)

✔ Accessible from anywhere (inside and outside the package).

- If a class, method, or variable is public, it can be accessed from anywhere, even from different packages (as long as it is imported).

Inside Package (mypackage/MyClass.java)

```
package mypackage;

public class MyClass {
   public void display() {
      System.out.println("Public method: Accessible
everywhere!");
   }
}
```

Outside Package (Test.java)

```
import mypackage.MyClass;

public class Test {
   public static void main(String[] args) {
      MyClass obj = new MyClass();
      obj.display();  // ✅ Works fine
   }
}
```

Public members are accessible everywhere, even from different packages.

Private (private)

❌ Accessible only within the same class.

- private methods and variables cannot be accessed outside the class—not even in the same package.
- Classes cannot be private, only members inside a class can be.

```
package mypackage;

public class MyClass {
   private void display() {
      System.out.println("Private method: Only inside this
class.");
   }

   public void test() {
      display();  // ✅ Works inside the same class
   }
}
```

Inside Package (mypackage/MyClass.java)

```
package mypackage;

public class Test {
   public static void main(String[] args) {
      MyClass obj = new MyClass();
      // obj.display();  // ❌ ERROR: Cannot access private
method
   }
}
```

Outside the Class (mypackage/Test.java)

❌ **Private members are not accessible outside the class, even in the same package.**

Protected (protected)

✔ Accessible within the same package and in subclasses (even in different packages).

- protected allows access inside the same package.
- Outside the package, it is only accessible through inheritance (subclassing).

```
package mypackage;

public class MyClass {
   protected void display() {
      System.out.println("Protected method: Accessible in the same package or via inheritance.");
   }
}
```

Inside Package (mypackage/MyClass.java)

```
package mypackage;

public class Test {
   public static void main(String[] args) {
      MyClass obj = new MyClass();
      obj.display();  // ✅ Works fine (same
package)
   }
}
```

Inside Same Package (mypackage/Test.java)

Outside Package (Only through Inheritance)

```
package anotherpackage;
import mypackage.MyClass;

public class SubClass extends MyClass {
   public static void main(String[] args) {
      SubClass obj = new SubClass();
      obj.display();  // ✅ Works (because of inheritance)
   }
}
```

4. Default (No Modifier)

❌ **Accessible only within the same package.**

- If no modifier is specified, the member/class has **package-private** access.
- It **cannot be accessed outside the package** (even via inheritance).

```
package mypackage;

class MyClass {  // No modifier = default access
   void display() {
      System.out.println("Default method: Accessible only in the
same package.");
   }
}
```

Inside Package (mypackage/MyClass.java)

```
package mypackage;

public class Test {
   public static void main(String[] args) {
      MyClass obj = new MyClass();
      obj.display();  // ✅ Works fine (same package)
   }
}
```

Inside Same Package (mypackage/Test.java)

Outside Package (anotherpackage/Test.java)

```
package anotherpackage;
import mypackage.MyClass;  // ❌ ERROR: MyClass is not public

public class Test {
    public static void main(String[] args) {
        MyClass obj = new MyClass();  // ❌ ERROR: Cannot access default class
        obj.display();  // ❌ ERROR
    }
}
```

❌ **Default members are not accessible outside the package.**

Practical 2:
Aim :To create “Bank Account Management System.” using the concept of Package and access modifiers.

Exercise Task:

1. Create a package bank and define a class BankAccount with different access modifiers.
2. Create a subclass Employee inside the same package to demonstrate **protected access**.
3. Create another package customer and try to access the methods from outside the package.

In a banking system, different information should be accessible at different levels.

- **Public:** Anyone should be able to check the bank name.
- **Private:** A customer's account balance and PIN should be private.
- **Protected:** Only bank employees (or subclasses) should access internal account details.
- **Default:** Internal calculations should be accessible only within the same package.

Unit 2 part 2

1. Inheritance
2. Interfaces
3. Super keyword & this keyword.
4. Method overriding
5. Runtime Polymorphism
6. Static & final keyword
7. Static vs Dynamic Binding
8. Default Methods
9. Abstract classes & methods
10. Is-A / Has-A relationships
11. Aggregation & Composition
12. Nested / Inner / Anonymous classes
13. StringBuffer, Tokenizer
14. Applets + Life Cycle + Security

1. Inheritence

- Inheritance & Code Reusability
  - Extending classes for code reusability
  - Usage of super keyword

Inheritance & Code Reusability in Java

Extending Classes for Code Reusability

- Allows a class to **inherit properties** and **behaviors** from another class.
- Reduces code duplication and promotes modularity.
- A child class (subclass) extends a parent class (superclass).

Usage of super Keyword

- Refers to the parent class from within the child class.
- Used to call the parent class constructor, methods, or access parent class variables.

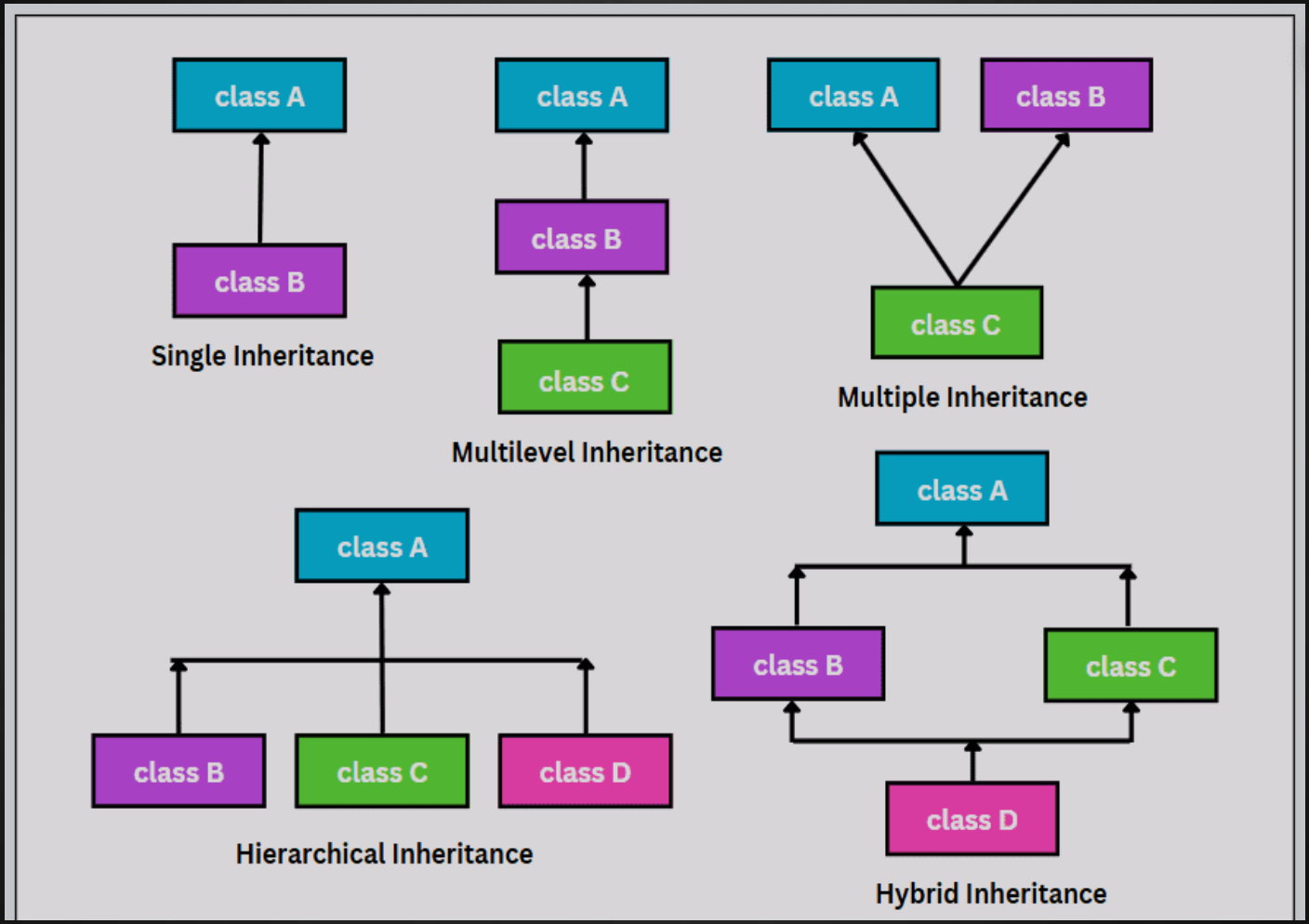
Method Overriding

- A child class provides a new implementation for a method defined in the parent class.
- Supports runtime polymorphism and improves flexibility.

Object Class

- The Object class is the parent class of all Java classes.
- Provides common methods like toString(), equals(), and hashCode().

Inheritence



Inheritance is a fundamental concept in **Object-Oriented Programming (OOP)** that allows a **child class (subclass)** to acquire the properties and behaviors (methods) of a **parent class (superclass)**.

It promotes **code reusability** and establishes a hierarchical relationship between classes.

Key Benefits of Inheritance

✅ Code Reusability – Avoids redundancy by reusing existing code.
✅ Method Overriding – Allows modifying inherited methods to fit specific needs.
✅ Extensibility – Enables adding new functionalities without modifying existing code.
✅ Polymorphism Support – Allows treating child classes as instances of their parent class.

Syntax

```
class ParentClass {
    // Parent class members (fields &
methods)
}

class ChildClass extends ParentClass {
    // Additional properties & methods
of child class
}
```

Types of Inheritance in Java

Java supports the following types of inheritance:

1. **Single Inheritance** – A child class inherits from one parent class.
2. **Multilevel Inheritance** – A child class inherits from another child class, forming a hierarchy.
3. **Hierarchical Inheritance** – Multiple child classes inherit from a single parent class.
4. **Multiple Inheritance (via Interfaces)** – Java does not support multiple inheritance with classes, but it can be achieved using interfaces.
5. **Hybrid Inheritance** – A combination of multiple types of inheritance using interfaces.

1. Single Inheritance

```
// Parent class
class Animal
{
    void eat() {
        System.out.println("This animal is eating.");
    }
}

// Child class inheriting from Animal
class Dog extends Animal
{
    void bark() {
        System.out.println("Dog is barking.");
    }
}
```

```
// Main class
public class SingleInheritanceExample
 {
    public static void main(String[] args)
 {
        Dog myDog = new Dog();
        myDog.eat();  // Inherited method
        myDog.bark(); // Child class method
    }
}
```

2. Multilevel Inheritance

```
// Grandparent class
class Animal
 {
    void eat()
   {
        System.out.println("This animal is eating.");
    }
}
// Parent class inheriting from Animal
class Mammal extends Animal
{
    void walk() {
        System.out.println("Mammal is walking.");
    }
}
// Child class inheriting from Mammal
class Dog extends Mammal
 {
    void bark() {
        System.out.println("Dog is barking.");
    }
}
```

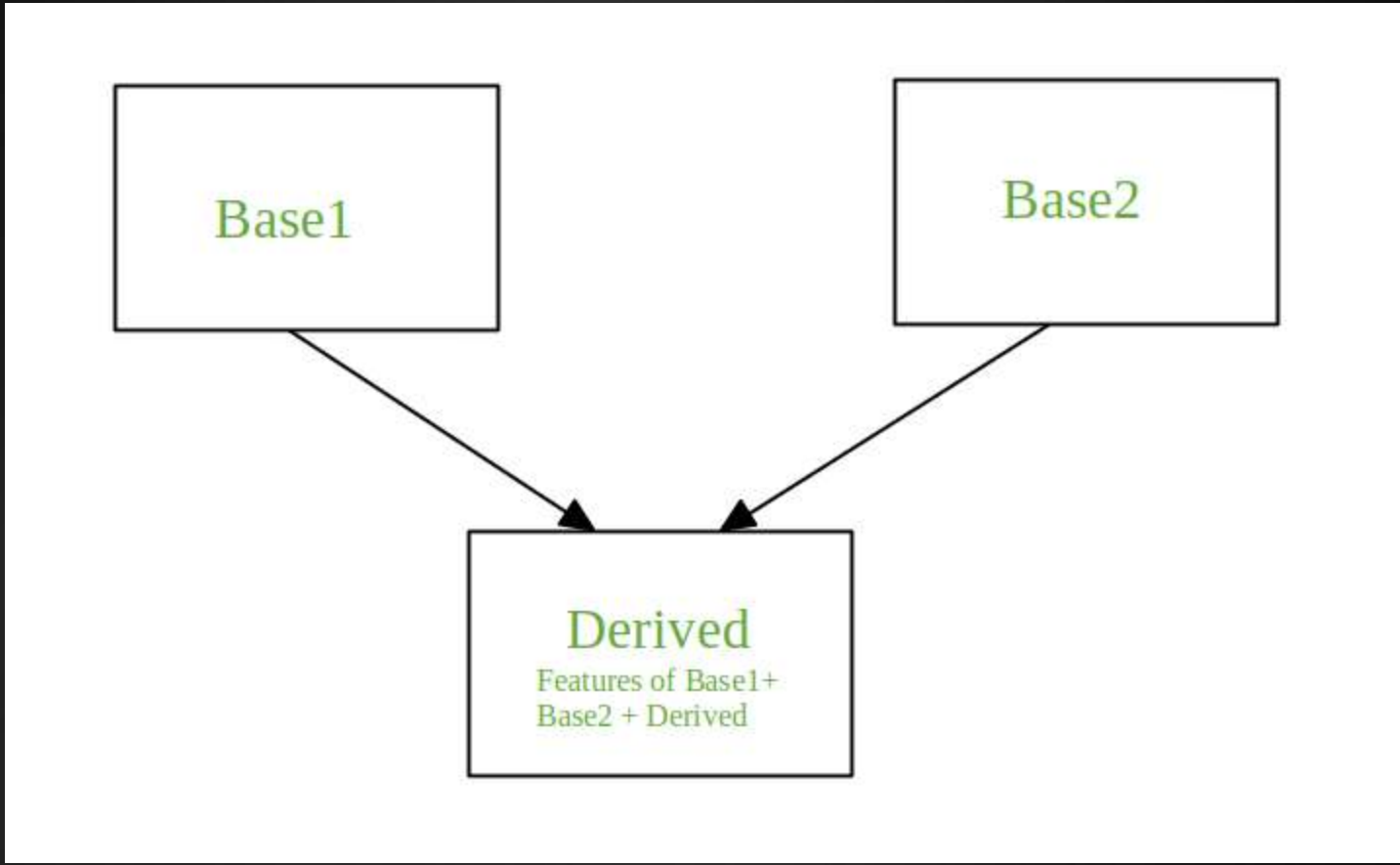
```
// Main class
public class MultilevelInheritanceExample
{
    public static void main(String[] args)
{
        Dog myDog = new Dog();
        myDog.eat();  // Inherited from Animal
        myDog.walk(); // Inherited from Mammal
        myDog.bark(); // Child class method
    }
}
```

3. Hierarchical Inheritance

```
// Parent class
class Animal {
    void eat() {
        System.out.println("This animal is
eating.");
    }
}
// Child class 1
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking.
    }
}
// Child class 2
class Cat extends Animal {
    void meow() {
        System.out.println("Cat is
meowing.");
    }
```

```
// Main class
public class
HierarchicalInheritanceExample {
    public static void main(String[] args)
{
        Dog myDog = new Dog();
        myDog.eat();  // Inherited from
Animal
        myDog.bark(); // Dog-specific
method

        Cat myCat = new Cat();
```

Multiple inheritence



Problem in Multiple Inheritance: **(Diamond Problem)**

In languages like C++, multiple inheritance allows a class to inherit from multiple parent classes.
However, this creates an issue called the **Diamond Problem** when two parent classes have methods with the same name.

```
// Two parent classes with the same method
class A {     void show() {        System.out.println("Class A");     }}
class B {     void show() {        System.out.println("Class B");     }}

// Trying to inherit from both classes

class C extends A, B { // ❌ Not allowed in Java
   // Ambiguity: Which show() method should C inherit?
}
```

2. Interface

Definition of Interface in Java

An **interface** in Java is a **blueprint** of a class that contains only **abstract methods (methods without a body)** and **constants (static final variables)**. It is used to achieve **abstraction** and **multiple inheritance** in Java.

Key Characteristics of Interfaces

1. **Only Abstract Methods** – Before Java 8, interfaces could only contain abstract methods. From Java 8 onwards, they can have default and static methods.
2. **No Constructors** – Interfaces cannot have constructors since they cannot be instantiated.
3. **Implementation via Classes** – A class implements an interface using the implements keyword.
4. **Multiple Inheritance Support** – A class can implement multiple interfaces, solving the diamond problem found in multiple inheritance.

Syntax

```
interface Animal
        {
                void makeSound();  // Abstract method (no body)
        }

class Dog implements Animal
        {
                public void makeSound() {  // Implementing method
                System.out.println("Dog barks.");
                }
        }

public class InterfaceExample
{
    public static void main(String[] args)
        {
                Dog myDog = new Dog();
                myDog.makeSound();
        }
}
```

4. Multiple Inheritance (Using Interfaces)

```
// First interface
interface Animal {
    void eat();
}

// Second interface
interface Bird {
    void fly();
}

// Class implementing both interfaces
class Bat implements Animal, Bird
 {
    public void eat() {
        System.out.println("Bat is eating.");
    }
    public void fly() {
        System.out.println("Bat is flying.");
    }
}
```

```
// Main class
public class MultipleInheritanceExample {
    public static void main(String[] args) {
        Bat myBat = new Bat();
        myBat.eat(); // Method from Animal interface
        myBat.fly(); // Method from Bird interface
    }
}
```

The **super keyword** in Java is a **reference variable** used to refer to the parent class (superclass).

Key Advantages of super Keyword

- Avoids ambiguity when methods and variables are overridden.
- Ensures parent class constructor runs first to initialize inherited properties.
- Facilitates code reusability by calling parent class methods inside child classes
- Helps resolve name conflicts between parent and child class variables.

1. Single Inheritance (Using super)

```
// Parent class
class Animal {
    Animal() {
        System.out.println("Animal constructor called.");
    }

    void eat() {
        System.out.println("Animal is eating.");
    }
}

// Child class using super
class Dog extends Animal {
    Dog() {
        super(); // Calls parent constructor
        System.out.println("Dog constructor called.
    }

    void bark() {
        System.out.println("Dog is barking.");
    }
}
```

```
//Main class
public class SingleInheritanceSuper {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.eat();  // Calls parent method
        myDog.bark(); // Child class method
    }
}
```

2. Multilevel Inheritance (Using super)

```
// Grandparent class
class Animal {
    Animal() {        System.out.println("Animal constructor called.");    }
    void eat() {        System.out.println("Animal is eating.");    }
}
// Parent class using super
class Mammal extends Animal {
    Mammal() {        super(); // Calls Animal constructor
        System.out.println("Mammal constructor called.");
    }
    void walk() {        System.out.println("Mammal is walking.");    }
}
// Child class using super
class Dog extends Mammal {
    Dog() {
        super(); // Calls Mammal constructor
        System.out.println("Dog constructor called.");
    }
    void bark() {   System.out.println("Dog is barking.");
    }
}
```

```
// Main class
public class MultilevelInheritanceSuper {
    public static void main(String[] args) {
        Dog myDog = new Dog();
        myDog.eat();  // Calls Animal method
        myDog.walk(); // Calls Mammal method
        myDog.bark(); // Dog method
    }
}
```

3. Super keyword
&
this keyword.

Super Keyword

In **Java**, the **super keyword** is used to refer to the **parent class (superclass) object**.
It helps you access parent class **variables, methods, and constructors** when they are hidden or overridden in the child class.
Main Uses of super Keyword

1.Access Parent Class Variable

```java
class Parent {
    int x = 10;
}

class Child extends Parent {
    int x = 20;

    void show() {
        System.out.println(super.x); // parent variable
        System.out.println(x);       // child variable
    }
}
```

Output

```
10
20
```

2. Call Parent Class Method (when overridden)

```java
class Parent {
    void show() {
        System.out.println("Parent method");
    }
}

class Child extends Parent {
    void show() {
        System.out.println("Child method");
    }

    void display() {
        super.show();
    }
}
```

3. Calls parent version of method.

```java
class Parent {
    Parent() {
        System.out.println("Parent constructor");
    }
}

class Child extends Parent {
    Child() {
        super();   // calls parent constructor
        System.out.println("Child constructor");
    }
}
```

This keyword

In **Java**, the **this keyword** is a reference variable that refers to the **current object** (the object that is calling the method or constructor).

Use it when you want to clearly refer to the current class's instance variables or methods.

Main Uses of this Keyword in Java

1. To Differentiate Instance Variables from Parameters

When constructor or method parameters have the same name as instance variables.

```
class Student {
    int id;
    String name;

    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}
```

2. To Call Current Class Method

```
class Demo {
    void show() {
        System.out.println("Hello");
    }

    void display() {
        this.show();
    }
}
```

this.show() calls method of current object.

3. To Call Current Class Constructor (Constructor Chaining)

```
class Demo {
    Demo() {
        this(10);
        System.out.println("Default constructor");
    }

    Demo(int x) {
        System.out.println("Parameterized constructor");
    }
}
```

Must be first line inside constructor.

4. To Pass Current Object as Argument

```
class Demo {
    void print(Demo d) {
        System.out.println("Object received");
    }

    void call() {
        print(this);
    }
}
```

5. To Return Current Object

Used in method chaining.

```
class Test {
    Test get() {
        return this;
    }
}
```

4. Method Overriding in Java

Method Overriding allows a subclass to provide a specific implementation of a method that is already defined in its parent class.

Rules for Method Overriding

1. **Same Method Signature** - The method in the child class must have the same name, return type, and parameters as the parent class.

2. **Inheritance is Required**

3. **Access Modifier Restrictions**:
   - The overridden method cannot have a more restrictive access modifier than the parent class method.

   - Example: If the parent method is protected, the overridden method cannot be private.

4. **Cannot Override Final Methods**
5. **Static Methods Cannot Be Overridden**
6. **Use of super Keyword -** The super keyword can be used to call the parent class method inside the child class.

```java
// Parent class
class Animal {
    void makeSound() {   System.out.println("Animal makes a sound");     }
}
// Child class
class Dog extends Animal {
    @Override
    void makeSound() {   System.out.println("Dog barks");     }
}

// Main class
public class MethodOverridingExample
{
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        myAnimal.makeSound();  // Calls the parent class method

        Dog myDog = new Dog();
        myDog.makeSound();  // Calls the overridden method in Dog class

        Animal myPet = new Dog();
        myPet.makeSound();  // Calls Dog's method (Runtime Polymorphism)
    }
}
```

o/p:
Animal makes a sound
Dog barks
Dog barks

@**Override annotation** (optional but recommended) helps catch errors if method names or signatures do not match.

5. Run time polymorphism

In runtime polymorphism, overridden methods are resolved based on the **object created**, not the reference variable.

```
Animal a = new Dog();   // parent reference, child object
```

Conditions Required

Runtime polymorphism occurs when:

1. Inheritance exists
2. Method is overridden
3. Parent reference points to child object
4. Method is non-static (static methods are not polymorphic)

6. Final and static

- If makeSound() in Animal were declared as **final**, the Dog class would not be able to override it.
- If makeSound() in Animal were **static**, method overriding would not work, and
method resolution would be based on the reference type, not the object type.

Final

```
class Animal {
    final void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    // Attempting to override final method will cause a compilation error!
    /*
    void makeSound() {
        System.out.println("Dog barks");
    }
    */
}

public class FinalMethodExample {
    public static void main(String[] args) {
        Animal myAnimal = new Animal();
        myAnimal.makeSound();  // Works fine

        Dog myDog = new Dog();
        myDog.makeSound();  // Calls inherited final method from Animal
    }
}
```

The final keyword prevents method overriding to ensure the method's behavior remains unchanged in subclasses

static

```
class Animal {
    static void makeSound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    static void makeSound()
    {
        System.out.println("Dog barks");
    }
}

public class StaticMethodExample {
    public static void main(String[] args)
 {
        Animal myAnimal = new Animal();
        myAnimal.makeSound();  // Calls Animal's static method

        Dog myDog = new Dog();
        myDog.makeSound();  // Calls Dog's static method

        Animal myPet = new Dog();
        myPet.makeSound();  // Calls Animal's static method, NOT Dog's
    }
}
```

o/p
Animal makes a sound
Dog barks
Animal makes a sound

- Static methods are resolved at compile-time (method hiding) rather than at runtime (method overriding).

- Animal myPet = new Dog();
still calls Animal.makeSound(), because method resolution is based on the reference type (Animal), not the actual object type (Dog).

Static vs Dynamic Binding

Static Binding (Early Binding)

Definition:
Method call is resolved at **compile time**.

Based on:
Reference type + method signature.

Used for:
- `static` methods
- `final` methods
- `private` methods
- Method overloading

Features:
- Faster (decided before program runs)
- No runtime polymorphism
- Method body is known at compile time

Dynamic Binding (Late Binding)

Definition:
Method call is resolved at **runtime**.

Based on:
Actual object type (not reference type).

Used for:
- Method overriding
- Runtime polymorphism

Features:
- Decided during execution
- Supports polymorphism
- Slightly slower than static binding

```
class Animal {
    void sound() {        System.out.println("Animal sound");    }
}
class Dog extends Animal {
    void sound() {        System.out.println("Dog barks");     }
}
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();   // runtime decision → Dog method dynamic binding
    }
}
```

```
class Calc {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
}// overloading is static binding
```

```
class Test {
    static void show() {      System.out.println("Static method");    }
    public static void main(String[] args) {
        Test.show();   // resolved at compile time
    }
}// static method call
```

Default Methods in Java

A **DEFAULT METHOD** IN JAVA IS A METHOD INSIDE AN **INTERFACE** THAT HAS A **CONCRETE IMPLEMENTATION** (I.E., IT CONTAINS A METHOD BODY).

IT WAS INTRODUCED IN **JAVA 8** TO ALLOW INTERFACES TO HAVE BEHAVIOR **WITHOUT BREAKING EXISTING IMPLEMENTING CLASSES**.

```
interface MyInterface
 {
   default void show() {      System.out.println("This is a default method in an interface.");
}
}

class MyClass implements MyInterface {}

public class Main
{
   public static void main(String[] args)
  {
      MyClass obj = new MyClass();
      obj.show(); // Calls the default method from MyInterface
   }
}
```

```
o/p:
This is a default method in an interface.
```

Here, MyClass ***does not override*** show(), but it still gets a working implementation from the interface.

7. Abstract class & abstract Methods

Abstract Class in Java

An **abstract class** is a class that:
Cannot be instantiated (no object can be created directly)
Is meant to be **inherited** by other classes

Can contain **abstract methods + normal methods**

Key Points
•Declared using `abstract` keyword
•Can have:
- abstract methods (no body)
- concrete methods (with body)

•Cannot create object directly
•Child class must implement abstract methods

Why Use Abstract Class?
•To provide a **common template**
•To force child classes to implement required behavior
•To support partial abstraction

```
abstract class Animal
  {
    abstract void sound();   // abstract method
    void eat() {System.out.println("Animal eats");   } // normal method
}

class Dog extends Animal
{
    void sound() {  System.out.println("Dog barks");    }
}

public class Test
{
    public static void main(String[] args)
   {
        Dog d = new Dog();
        d.sound();
        d.eat();
    }
}
```

Is-A / Has-A relationships

"Is-A" Relationship in Java (Inheritance)

The **"Is-A" relationship** in Java represents **inheritance**, where one class derives properties and behaviors from another class.

This means the child class **is a specialized version** of the parent class.

Key Points About Inheritance:

1. **Parent-Child Relationship**: The child class inherits attributes and behaviors (methods) from the parent class.
2. **Code Reusability**: Instead of rewriting the same code in multiple classes, we define common functionalities in the parent class and reuse them in child classes.
3. **Polymorphism Support**: Inheritance enables method overriding, which allows a child class to provide a specific implementation of a method already defined in the parent class.
4. **Extending Functionality**: A child class can have additional attributes and methods apart from the inherited ones.

Types of Inheritance in Java:

1. **Single Inheritance** – A child class inherits from a single parent class.
2. **Multilevel Inheritance** – A child class inherits from a parent class, which itself is a child of another class (forming a chain).
3. **Hierarchical Inheritance** – A single parent class has multiple child classes.
4. **Multiple Inheritance (Through Interfaces)** – Java does not support multiple inheritance with classes, but it can be achieved using interfaces.

Real-Life Examples of Inheritance in Java
Inheritance in Java follows an **"is-a" relationship**, meaning a child class is a specialized version of a parent class. Here are some real-world examples:

1. **Animal Kingdom (Hierarchical Inheritance)**

- Parent Class: Animal (Has common properties like breathing, eating)
- Child Classes:
  - Mammal (Has properties like warm-blooded)
  - Bird (Has properties like feathers, can fly)
  - Fish (Has properties like can swim, gills)

Example Relationship:
- A dog is a mammal → A mammal is an animal (Multilevel Inheritance).
- A sparrow is a bird → A bird is an animal (Hierarchical Inheritance).

Real-Life Examples of Inheritance in Java
Inheritance in Java follows an **"is-a" relationship**, meaning a child class is a specialized version of a parent class. Here are some real-world examples:

2. Vehicle Classification (Single Inheritance)
- Parent Class: Vehicle (Has properties like speed, fuel type)
- Child Class: Car (Inherits speed and fuel type but has additional properties like airbags, AC)

Example Relationship:
- A car is a vehicle.
- A truck is a vehicle.

Real-Life Examples of Inheritance in Java
Inheritance in Java follows an **"is-a" relationship**, meaning a child class is a specialized version of a parent class. Here are some real-world examples:

3. Employee Management System (Multilevel Inheritance)
- Parent Class: Employee (Has properties like name, ID, salary)
- Child Class: Manager (Inherits name, ID, and salary but has additional properties like team size)
- Grandchild Class: ProjectManager (Inherits everything from Manager but adds project name)

Example Relationship:
- A Project Manager is a Manager.
- A Manager is an Employee.

Real-Life Examples of Inheritance in Java
Inheritance in Java follows an **"is-a" relationship**, meaning a child class is a specialized version of a parent class. Here are some real-world examples:

4. Educational Institution (Hierarchical Inheritance)

- Parent Class: Person (Has properties like name, age)
- Child Classes:
  - Student (Has properties like roll number, grades)
  - Teacher (Has properties like subject specialization, salary)
  - Staff (Has properties like department, working hours)

Example Relationship:
- A student is a person.
- A teacher is a person.

Real-Life Examples of Inheritance in Java
Inheritance in Java follows an **"is-a" relationship**, meaning a child class is a specialized version of a parent class. Here are some real-world examples:

5. Banking System (Multiple Inheritance using Interfaces)

- Interface 1: LoanProvider (Has method provideLoan())
- Interface 2: AccountHolder (Has method deposit(), withdraw())
- Child Class: Bank (Implements both interfaces, meaning it provides loans and allows deposits/withdrawals)

Example Relationship:
- A bank is both a loan provider and an account holder.

Has-A relation

AGGREGATION & COMPOSITION

- The **"Has-A" relationship** in Java represents **composition or aggregation**, where one class contains an instance of another class as a field.
- Instead of inheriting properties (as in **"Is-A" relationship** through inheritance), a class **uses** another class as part of its functionality.

"Has-A" Relationship Types:

- **Composition (Strong Association)**

  If the contained object **cannot** exist independently without the container class

- **Aggregation (Weak Association)**

  If the contained object **can** exist independently of the container class

Real-Life Examples of "Has-A" Relationship:

1. Human and Heart (Composition - Strong Association)

- A Human has a Heart.
- The Heart **cannot exist** separately outside the Human.
- If the Human dies, the Heart stops functioning.

Example:

- A Human class contains a Heart object.
- The Heart is part of the Human and cannot function independently.

Real-Life Examples of "Has-A" Relationship:

2. Library and Books (Aggregation - Weak Association)

- A Library has Books.
- Books can exist separately outside the Library (can be sold, borrowed, or placed in another Library).

Example:

- The Library class contains a collection of Book objects.
- The Book class is independent and can be used elsewhere.

Real-Life Examples of "Has-A" Relationship:

3. House and Rooms (Composition - Strong Association)

- A House has Rooms.
- The Rooms cannot exist separately without the House.
- If the House is demolished, the Rooms also cease to exist.

Example:

- A House class contains multiple Room objects.
- Each Room belongs to a specific House.

Real-Life Examples of "Has-A" Relationship:

4. Company and Employees (Aggregation - Weak Association)

- A Company has Employees.
- Employees can exist separately (they can work for different companies).
- If the Company shuts down, Employees can still find jobs elsewhere.

Example:

- A Company class has a list of Employee objects.
- Employees are independent and not tightly coupled with the Company.

```
class Student
{
    String name;
    Student(String name) {        this.name = name;    }
}

class College
 {
    Student student;   // aggregation
    College(Student student) {  this.student = student;    }
    void show() {        System.out.println(student.name);  }
}
public class TestAggregation
{
    public static void main(String[] args)
    {
        Student s = new Student("Neha");   // created outside
        College c = new College(s);

        c.show();
    }
}
```

Aggregation
(Weak has-a — object created outside)
Example: Student belongs to College

```
class Engine
{
    Engine() {   System.out.println("Engine created");     }
}

class Car
{
    Engine e = new Engine();   // composition
}

public class TestComposition
{
    public static void main(String[] args)
    {
        Car c = new Car();
    }
}
```

Composition
(Strong has-a — object created inside)
Example: Engine inside Car

Difference Between "Is-A" and "Has-A" Relationship

Feature	"Is-A" Relationship (Inheritance)	"Has-A" Relationship (Composition/Aggregation)
Definition	One class is a subtype of another class.	One class contains another class as an instance variable.
Example	A `Car` is a `Vehicle`.	A `Car` has an `Engine`.
Achieved By	Using `extends` or `implements` keywords.	Using object references as attributes.
Code Reusability	High (child class reuses parent class methods).	Moderate (one class uses another class's functionalities).
Coupling	Strongly coupled (tight relationship).	Loosely coupled (more flexible).

Key Differences: Composition vs. Aggregation

Feature	Composition (Strong Association)	Aggregation (Weak Association)
Dependency	Strong (one object **cannot exist** without the other).	Weak (one object **can exist** independently).
Object Lifecycle	Contained object is **destroyed** when the container object is destroyed.	Contained object **can exist** even if the container object is destroyed.
Flexibility	Less flexible (high coupling).	More flexible (low coupling).
Implementation	Directly includes an object as an instance variable.	Uses references (object is passed through constructor or method).
Real-Life Example	**Car and Engine** – Engine cannot exist separately from Car.	**University and Professor** – Professors can work elsewhere.

Nested classes,
Inner classes,
Anonymous Inner classes,
String Buffer Class,
tokenizer

A nested class is a class defined within another class that can access the members of the outer class and is used to increase code readability, maintainability, and security.

- Example

```
class Outer
{
    class Inner
    {
        void show() { System.out.println("This is a nested class"); }
    }
}
```

Types of Nested Classes in Java

- 1) Static Nested Class
- 2) Inner Class
  - a) Member Inner Class
  - b) Local Inner Class
  - c) Anonymous Inner Class

Static Nested Class

- Declared with static keyword
- Does not require outer class object
- Can access only static members of outer class directly
- Cannot access non-static members directly
- Used for helper/utility classes related to outer class

```
class Outer
 {
      static int x = 100;

       static class Inner
       {
            void show() {   System.out.println("Value of x = " + x);     }
       }

   public static void main(String[] args) {
      Outer.Inner obj = new Outer.Inner();
      obj.show();
   }
}
```

OUTPUT:
Value of x = 100

Inner Classes in Java

An **Inner Class** is a non-static nested class defined inside another class.

It can access all members (including private) of the outer class.

Inner classes are of three types:

a) Member Inner Class
b) Local Inner Class
c) Anonymous Inner Class

a) Member Inner Class

- Defined at class level inside outer class
- No static keyword
- Can access all outer class members
- Requires outer class object

```
class Outer {

   // Member Inner Class (inside class)
   class Inner {
      void show() {  System.out.println("Member Inner Class");  }
   }
}

public class Test {
   public static void main(String[] args) {

      Outer o = new Outer();        // create outer object
      Outer.Inner i = o.new Inner(); // create inner object using outer object
      i.show();                     // call method
   }
}
```

```
/*
Explanation:
- Inner class is defined inside Outer class
- It needs outer object to be created

Output:
Member Inner Class
*/
```

b) Local Inner Class

- Defined inside a method
- Scope limited to that method only
- Cannot be accessed outside the method

```
class Outer {

    void display() {
                class Inner {   void show() {   System.out.println("Local Inner Class");    }       }
                Inner i = new Inner(); // object created inside method only
                 i.show();
    }
}

public class Test {   public static void main(String[] args) {
  new Outer().display(); // call method    }}
```

```
/*
Explanation:
- Inner class is defined inside a method
- Scope only inside display() method

Output:
Local Inner Class
*/
```

c) Anonymous Inner Class

- No class name
- Declared and instantiated at the same time
- Mostly used with interfaces or abstract classes
- Used for one-time implementation

```
interface A {
    void show();
}

public class Test
  {
   public static void main(String[] args)
     {

       // Anonymous Inner Class (no class name)
       A obj = new A() {
                                public void show()
                                 {     System.out.println("Anonymous Inner Class");    }
       };
       obj.show();
    }
}
```

```
/*
Explanation:
- No class name
- Implemented and instantiated at same time
- Used for one-time use

Output:
Anonymous Inner Class
*/
```

StringBuffer Class in Java

- **StringBuffer** is a class in Java used to create and manipulate **mutable (modifiable) strings**. Unlike String, objects of StringBuffer can be changed after creation without creating new objects.

- **Key Features**
  - Mutable (content can be changed)
  - Thread-safe (methods are synchronized)
  - Stored in heap memory
  - Better than String for frequent string modifications
  - Slower than StringBuilder (because of synchronization)

```
StringBuffer sb = new StringBuffer("Hello");
```

Method	Use
append()	adds text at end
insert()	inserts text at position
delete()	deletes part of string
reverse()	reverses string
replace()	replaces substring
capacity()	returns buffer capacity
length()	returns current length
charAt()	returns character at index

```
class Demo
 {
   public static void main(String[] args)
  {
      StringBuffer sb = new StringBuffer("Java");

      sb.append(" Programming");
      System.out.println(sb);

      sb.insert(4, " Core");
      System.out.println(sb);

      sb.reverse();
      System.out.println(sb);
   }
}
```

Java Programming
Java Core Programming
gnimmargorP eroC avaJ

String vs StringBuffer

- String → immutable
- StringBuffer → mutable + thread-safe

String Tokenizer

- **StringTokenizer** is a class in Java used to **break a string into smaller parts called tokens** based on a delimiter (such as space, comma, etc.).
- It is available in the java.util package.

Method	Use
hasMoreTokens()	checks if more tokens exist
nextToken()	returns next token
countTokens()	returns number of tokens

StringTokenizer()

```
import java.util.StringTokenizer;

class Demo {
    public static void main(String[] args)
     {
       String str = "Java is very easy";
       StringTokenizer st = new StringTokenizer(str);

       while (st.hasMoreTokens()) {
           System.out.println(st.nextToken());
       }
    }
}
```

```
Java
is
very
easy
```

Applets + Life Cycle + Security

Definition

- A **Java Applet** is a small Java program that runs inside a web browser or applet viewer.
- It is embedded in an HTML page and executed by the Java Virtual Machine (JVM) on the client side.
- Applets are mainly used for graphical and interactive web applications.

Applet class belongs to:

java.applet.Applet

Object
↓
Component → makes it visible on screen
↓
Container → allows adding buttons, fields etc.
↓
Applet → makes it run inside browser

Applet inherits basic object behavior from Object, GUI capabilities from Component, component-holding ability from Container, and browser execution features from Applet class itself.

1. `java.lang.Object` — Root Class

•This is the **topmost class in Java**
•Every Java class automatically extends `Object`
•Provides basic methods:
Examples:
•`toString()`
•`equals()`
•`hashCode()`
•`getClass()`
So Applet also gets these basic methods.

2. `java.awt.Component` — GUI Element

•Adds **graphical behavior**
•Makes applet a visible GUI element
Gives features like:
•Drawing on screen
•Size & position
•Colors & fonts
•Event handling support
Examples:
`setSize()`
`setBackground()`
`setForeground()`
`repaint()`
Because of Component, applet can appear on screen.

3. `java.awt.Container` — Can Hold Components

•A Container can **hold other GUI components**
Examples:
•Buttons
•Labels
•TextFields
•Panels
Methods gained:
`add()`
`remove()`
`setLayout()`
Because of Container, you can put buttons and controls inside an applet.

4. `java.applet.Applet` — Browser-Runnable Program

•Final level — adds **applet-specific behavior**
Adds:
•Life cycle methods:
- `init()`
- `start()`
- `stop()`
- `destroy()`

•Browser interaction support
•Applet context & parameters

Applet Life Cycle

Applet execution is controlled by the browser through predefined methods.

Methods of Life Cycle

1. init()

- Called once when applet loads
- Used for initialization

2.start()

- Called after init()
- Called again when page becomes active
- Starts threads/animations

3. paint(Graphics g)

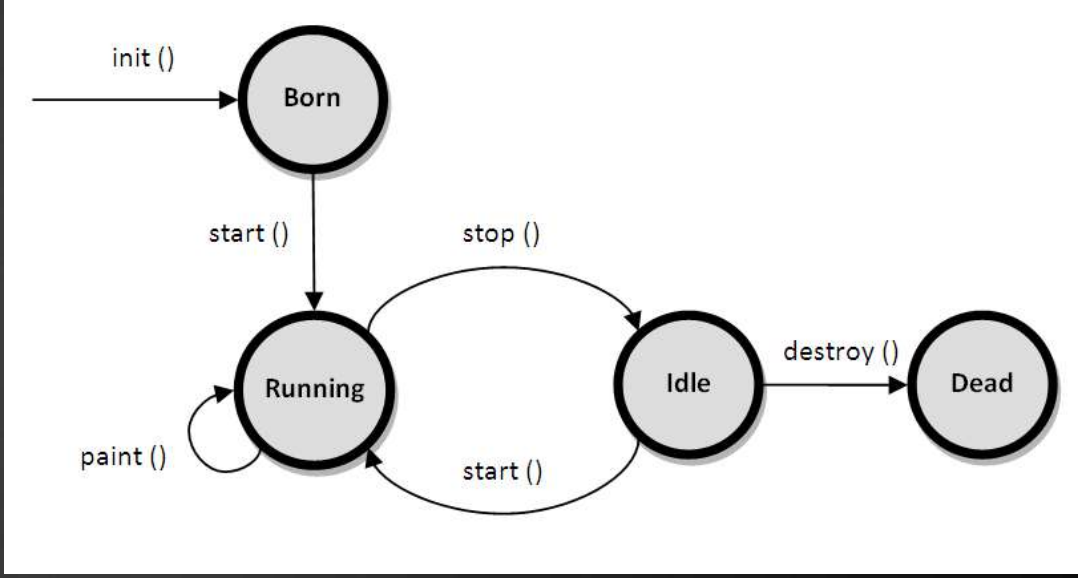
- Used for drawing output
- Called many times

4. stop()

- Called when page becomes inactive
- Stops background tasks

5.destroy()

- Called once before applet is removed
- Cleanup resources



```
import java.applet.Applet;
import java.awt.Graphics;

public class HelloApplet extends Applet
 {
    public void paint(Graphics g)
     {
        g.drawString("Hello World", 50, 50);
    }
}
```

```
<applet code="HelloApplet.class" width="300" height="150">
</applet>
```

RUN USING :

appletviewer file.html

Applet Security (Sandbox Model)
Applets run in a **sandbox environment** for safety.

Restrictions
- Cannot read/write local files
- Cannot execute system commands
- Cannot access OS resources
- Cannot connect to other servers (only origin server)
- Cannot load native libraries

Signed Applets
- Digitally signed applets
- User permission required
- Can access local resources
- Risk if malicious code is trusted

Limitations of Applets
- Require browser plugin
- Security risks
- Slow startup
- Not supported by modern browsers
- Mostly obsolete now